MINI-RELATÓRIO

Aplicação de Estrutura de Dados na Otimização da Aplicação

Integrantes: Eric Carvalho, Gabriel Vaz, Douglas Macario e Gustavo Melo
Turma / Disciplina: ADS 2 / Estrutura de Dados
Professor(a): Eduardo Savino
Data: 09/11/2026

1. Contexto

A aplicação desenvolvida pelo grupo é uma plataforma web de acompanhamento de alunos, com três perfis de acesso: Administrador, Professor e Responsável. O frontend foi feito em React (Vite) e o backend em Node.js. Os dados dos alunos e dos acompanhamentos são carregados em memória como arrays de objetos, e as telas precisam filtrá-los, agrupá-los e exibi-los conforme a interação do usuário. Por isso, a escolha correta da estrutura de dados e da forma de percorrê-la influencia diretamente a eficiência e a clareza do código.

2. Estruturas de dados aplicadas

A tabela resume as estruturas utilizadas, onde aparecem no projeto e qual o ganho obtido.

Estrutura	Onde foi usada	Por que foi escolhida	O que otimiza
Set	Responsavel.jsx (disciplinas e anos do aluno)	Não permite valores repetidos	Elimina laços de comparação manual; O(n) em vez de O(n²)
Array de objetos + filter em cadeia	Responsavel.jsx (acompanhamentos)	Permite reduzir a lista em etapas	Cada filtro trabalha sobre um conjunto menor de dados
Array de objetos + filter combinado	Administrador.jsx (lista de alunos)	Une vários critérios em uma só condição	Uma única passada pelo array, sem alterar a lista original

3. Trechos de código e como otimizam a aplicação

Trecho 1: Set para valores únicos (Responsavel.jsx)

```
const disciplinasDoAluno = [
  ...new Set(
    acompanhamentosPublicados.map(
      (item) => item.disciplina
    )
  )
]

const anosDoAluno = [
  ...new Set(
    acompanhamentosPublicados.map(
      (item) => item.ano
    )
  )
]
```

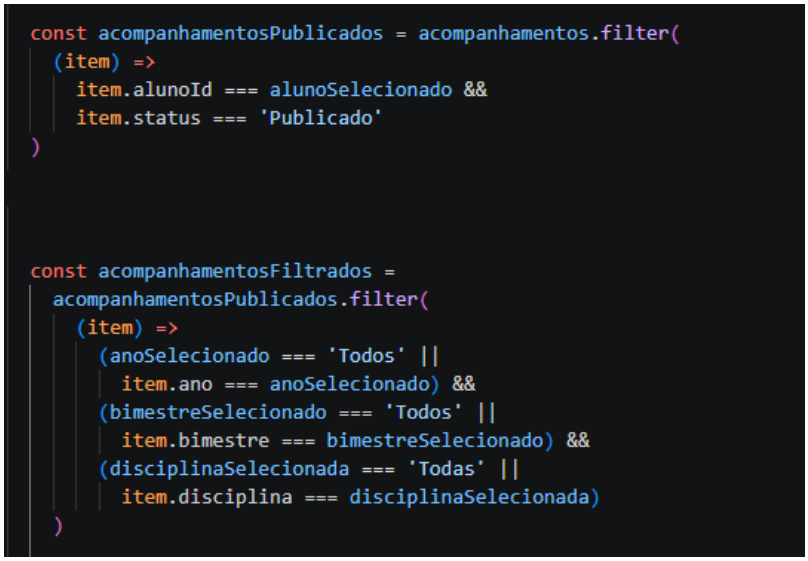
Os acompanhamentos publicados de um aluno repetem disciplinas e anos várias vezes. Para montar as opções dos filtros, o método .map() extrai o valor desejado de cada item e o Set elimina automaticamente as repetições. O spread [...] converte o Set de volta em array para uso no seletor da tela.

Otimização: sem o Set, seria necessário um laço que, a cada item, verificasse com .includes() se o valor já existe na lista, o que gera comparações repetidas ($O(n^2)$ no pior caso). Com o Set, a verificação de existência tem custo médio constante e a remoção de duplicatas fica em O(n), com menos código e menos chance de erro. O código real do projeto é o do print acima; para comparar, o bloco abaixo mostra como ficaria sem o Set.

EXEMPLO ILUSTRATIVO (não faz parte do código do projeto):

```
// Abordagem manual, apenas para comparação
const disciplinas = [];
acompanhamentos.forEach((item) => {
  if (!disciplinas.includes(item.disciplina)) {
    disciplinas.push(item.disciplina);
  }
});
```

Trecho 2: Filtros em cadeia sobre array de objetos (Responsavel.jsx)



```
const acompanhamentosPublicados = acompanhamentos.filter(
  (item) =>
    item.alunoId === alunoSelecionado &&
    item.status === 'Publicado'
)



const acompanhamentosFiltrados =
  acompanhamentosPublicados.filter(
    (item) =>
      (anoSelecionado === 'Todos' ||
        item.ano === anoSelecionado) &&
      (bimestreSelecionado === 'Todos' ||
        item.bimestre === bimestreSelecionado) &&
      (disciplinaSelecionada === 'Todas' ||
        item.disciplina === disciplinaSelecionada)
  )
```

Primeiro, acompanhamentosPublicados reduz o array geral apenas aos registros do aluno selecionado com status “Publicado”. Em seguida, acompanhamentosFiltrados aplica os filtros de ano, bimestre e disciplina somente sobre esse subconjunto. A opção “Todos/Todas” ignora o critério por meio de curto-circuito (||).

Otimização: como cada etapa parte de uma lista já reduzida, o volume de dados processado diminui a cada filtro, e os arrays originais não são modificados, o que permite reaproveitá-los em outras partes da tela.

Trecho 3: Filtro combinado em uma única passada (Administrador.jsx)

```
const alunosFiltrados = alunos.filter((aluno) => {
  const nomeValido = aluno.nome
    .toLowerCase()
    .includes(filtros.aluno.toLowerCase())

  const disciplinaValida =
    !filtros.disciplina ||
    aluno.disciplina === filtros.disciplina

  const statusValido =
    !filtros.status ||
    aluno.status === filtros.status

  return nomeValido && disciplinaValida && statusValido
})
```

No painel do Administrador, três critérios (nome, disciplina e status) são verificados dentro de um único .filter(), que devolve um novo array apenas com os alunos que atendem a todos eles.

Otimização: em vez de três filtragens separadas, o array é percorrido uma só vez (O(n)). Os critérios opcionais usam !filtros.disciplina || ..., que dispensa a comparação quando nenhum valor foi escolhido, e a lista original permanece intacta, sem necessidade de recarregar o arquivo de dados a cada filtro.

4. Conclusão

A aplicação dos conhecimentos de estrutura de dados no projeto se concentrou em escolher a estrutura adequada para cada necessidade: o Set para garantir valores únicos, e arrays de objetos com filtros encadeados ou combinados para reduzir e consultar dados com poucas passadas. Isso resultou em menos comparações desnecessárias, código mais curto e legível, e preservação dos dados originais.

Melhoria futura: o uso de useMemo para evitar recalcular os filtros a cada renderização e de um Map para buscas por identificador poderia reduzir ainda mais o processamento. Essa melhoria não está implementada na versão atual.